

임기호

lasid84@gmail.com

GitHub

복잡한 업무 흐름을 안정적인 서버 경계로 바꿉니다

운영 시스템을 API·Queue·Worker·Data 경계로 분리하고, 외부 연동과 장애·배포까지 운영해 온 서버·플랫폼 엔지니어입니다.

15 → 1

레거시 시스템 통합

1,000만+

데이터 이관

월 1,227h

반복 작업 절감 산정치

10분 → 5초

외부 정보 조회

01 · CAREER JOURNEY

주요 경력

제조·물류 시스템을 개발하고 운영해 온 경험을 시간순으로 정리했습니다.

2010-2015

ILJIN Global

MES·ERP 업무 시스템 개발

생산 데이터 수집과 MES·ERP 업무 시스템을 개발하며 제조 현장의 흐름을 익혔습니다.

2016-2021

ILJIN Global

국내외 공장 MES 고도화

노후 클라이언트를 현대화하고 국내외 공장의 추적성과 실시간 ERP 연계를 구축했습니다.

2022-2023

KWE Korea

물류 업무 자동화와 데이터 전환

반복 입력, 외부 연동, 여러 DB의 데이터 이관을 검증 가능한 자동화 흐름으로 바꿨습니다.

2023-현재

KWE Korea

통합 업무 플랫폼 개발 및 운영

15종 운영 시스템을 분석하고 웹·API·Worker·데이터 경계를 갖춘 단일 플랫폼으로 전환하고 있습니다.

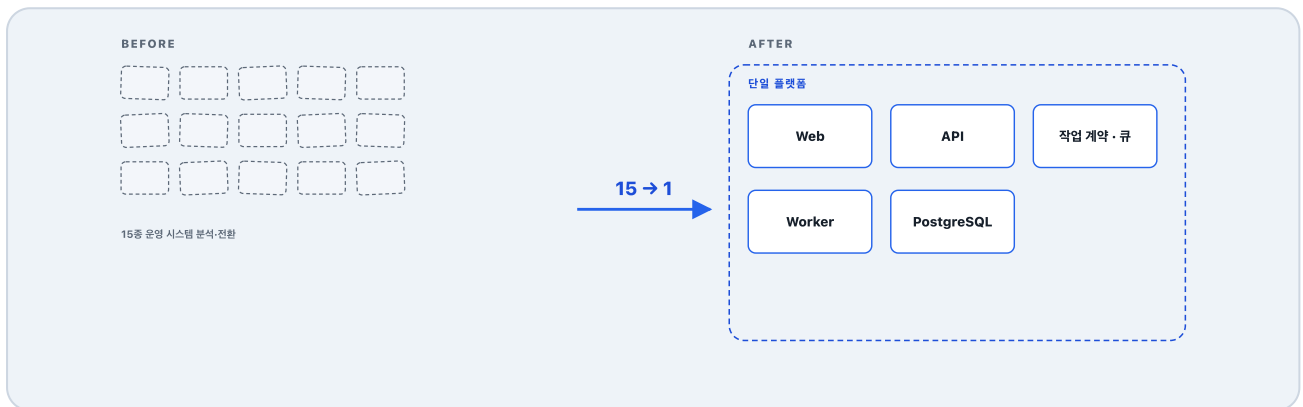
통합 업무 플랫폼 개발

분산되어 있던 운영 시스템을 웹·API·비동기 실행·외부 연동·데이터·배포 경계로 나누어 하나의 플랫폼으로 전환했습니다.

구현 범위

15종 운영 시스템 분석·전환	90+ 업무 화면 구현	5개 업무 도메인 분리	8종 외부 시스템 연동	4단계 API→Queue→Worker →API 내부 실행	변경 앱만 CI/CD 빌드·배포
----------------------------	------------------------	------------------------	------------------------	--	-----------------------------

전환 전후

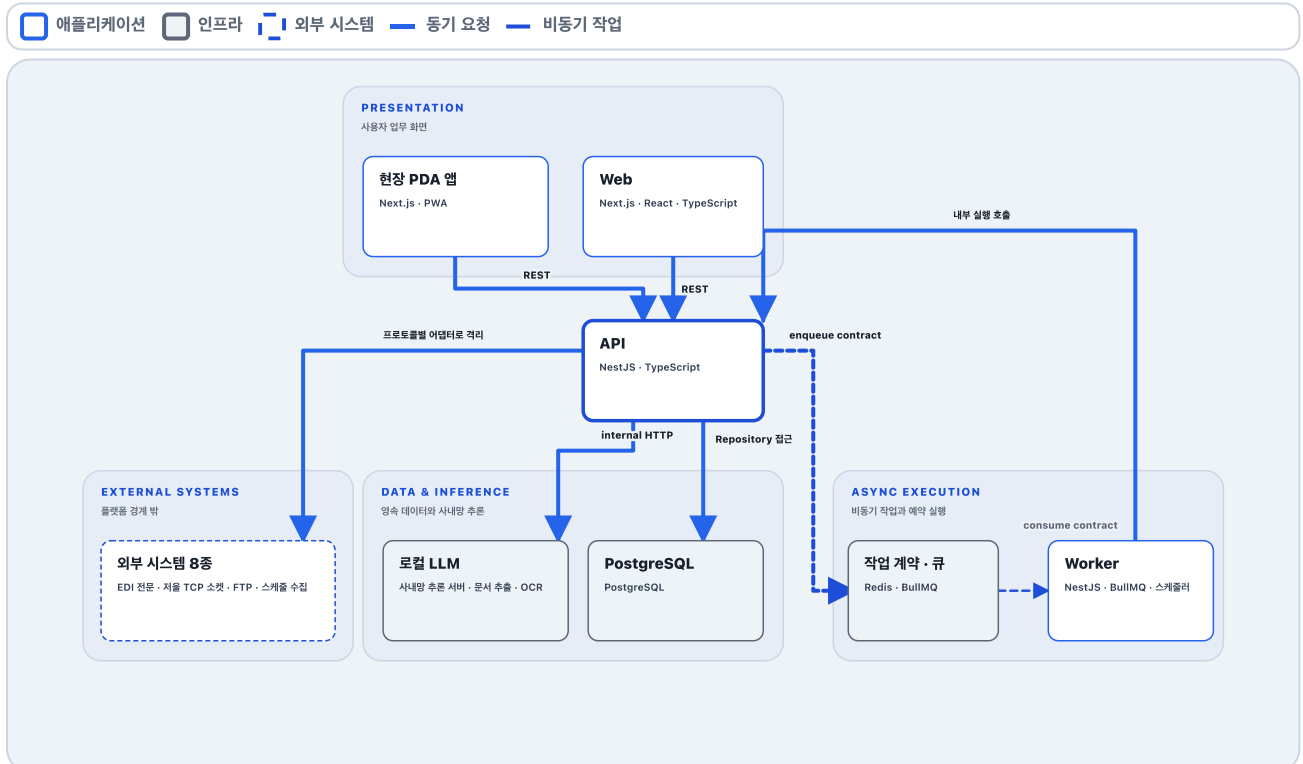


Oracle·MySQL·ODBC로 흩어져 있던 데이터는 DB 구조를 새로 설계해 PostgreSQL 하나로 통합했습니다. 1,000만 건 이상을 이관하면서 원본과 대상 양쪽에서 교차 검증했습니다.

시스템 구성

- 도메인 로직은 API에 모았습니다.
- Worker는 업무 로직을 갖지 않고 API의 같은 유스케이스를 호출합니다.
- 오래 걸리거나 실패할 수 있는 작업은 큐와 Worker로 뺐습니다.
- 프로토콜이 제각각인 외부 연동은 어댑터 안에 가렸습니다.

구성요소를 누르면 각 판단의 구현과 계약을 볼 수 있습니다.

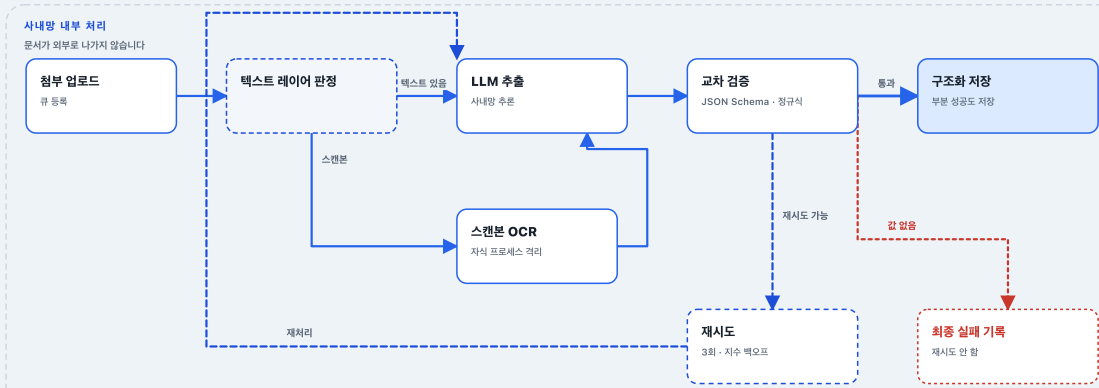


문서 처리

사내망 안에서만 처리되는 문서 추출 파이프라인입니다. Worker 동시성은 프로세스당 1건, 운영 레플리카 2개라 서버 전체 최대 2건입니다. 큐 등록이 실패해도 첨부 업로드는 실패시키지 않고 경고 로그로 남깁니다.

렌더 해상도는 실물 문서로 반복 측정해 정했습니다. 300dpi에서 나던 오인식이 400dpi에서 사라졌고 처리 시간은 거의 같았습니다.

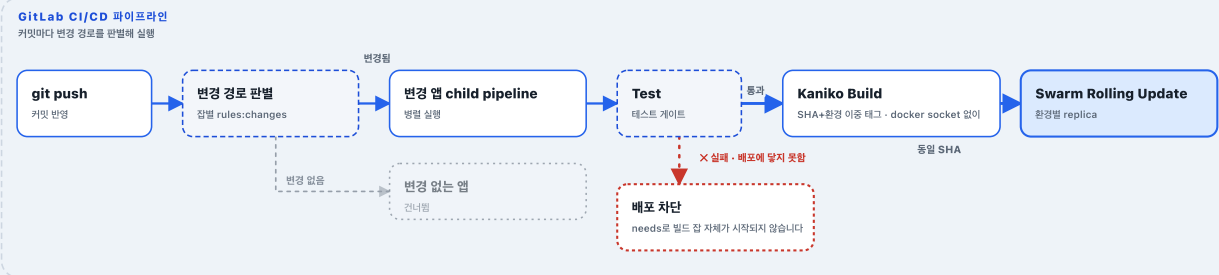
반복 평가 결과를 저장해 변경 전후를 비교합니다 (evaluation harness).



동그란 화살표는 순차 흐름, 주황 점선은 재시도 가능 실패, 얇은 점선은 관찰용 기록입니다.

배포

테스트를 통과하지 못한 변경은 그림의 구조상 배포에 물리적으로 닿지 않습니다. 브랜치에 따라 development·staging·production으로 갈리고, replica는 development 1개, staging·production 각 2개입니다.



실패 경로는 배포 차단 노드에서 끊어지고 어떤 배포 노드로도 이어지지 않습니다. 흐린 노드는 변경이 없어 건너뛴 앱입니다.

본사 운영 시스템 입력 자동화

앞 섹션 플랫폼이 연동하는 외부 시스템 8종 가운데 하나입니다. 화면이 쓰던 호출 규칙을 알아내 업무 단위로 다시 묶었습니다.

문제

다섯 부서가 본사 글로벌 운영 시스템 화면에서 건마다 같은 절차를 반복했습니다. 많게는 한 업무를 하루에 백 번 넘게 반복했습니다.

제약

본사 시스템이라 연동 API도 자동화 도구도 열려 있지 않았습니다. 사람이 화면을 여는 것 말고는 들어갈 경로가 없었습니다.

접근

개발자 도구로 화면이 주고받는 요청과 응답을 뜯어, 어떤 요청을 어떤 순서로 보내야 하는지 알아냈습니다. 응답 스키마에 맞춰 저장 구조도 설계했습니다.

75개 외부 호출

화면이 보내던 요청을 하나씩 그대로 재현합니다.

12종 업무 워크플로

사람이 처리하던 업무 한 건이 호출 한 번이 되도록 묶었습니다.

항공 운송장 갱신 한 건

사람이 화면을 아홉 번 오가야 끝나던 일입니다. 조회 여섯 번, 재계산 두 번, 저장 한 번입니다.

도입 전 부서별 처리 건수를 기준으로 산정한 월 절감 예상치입니다.

부서	기능	월 절감(시간)	월 처리량
항공수출	운임 코드·원가 자동 업로드	250	3,000건
항공수출	운임 청구금액 이상 자동 검출	100	3,000건
항공수출	항공 운송장 확정 처리	125	1,500건
항공수출	출하 자동 마감	80	500건
항공수출	전자문서 자동 전송	40	500건
항공수입	해외 정산 운임 자동 계산·업로드	165	1,000건
항공수입	국내 정산 운임 자동 업로드	230	1,400건
항공수입	출발지 부대비용 확인	70	2,000건
해운수입	운임 코드·원가 업로드	80	1,000건
경리	운임 코드·원가 업로드	80	1,000건
전산	환율 자동 입력	7	22회
합계 · 11종 · 5개 부서		1,227	

기록

이틀 동안 운영 API가 다섯 번 죽었다

원인을 세 번 짚었고 두 번 틀렸다. 로그 줄 수, 설치된 버전, 조회 행수를 직접 세서 잡았다.

커널 OOM 6~10GiB · 조회 54,565행 · 반복 측정 480회

[velog에서 읽기](#)

Next.js + BFF 인증 흐름 총정리

엑세스 토큰을 메모리에, 리프레시를 HttpOnly 쿠키에 두는 구성이다. 새로고침 복원과 401이 겹칠 때의 재발급까지 정리했다.

엑세스는 메모리 · 리프레시는 HttpOnly 쿠키 · 401에 single-flight 재발급

[velog에서 읽기](#)

CI/CD 적용기 2편

rsync과 PM2로 시작한 배포를 컨테이너와 Swarm으로 옮긴 과정이다. docker socket 없이 이미지를 굽고, 롤링 업데이트와 SHA 롤백까지 붙였다.

docker socket 없이 Kaniko 빌드 · start-first 롤링 업데이트 · SHA 태그로 롤백

[velog에서 읽기](#)

직접 만들고 운영한 제품



댓글 필터 · Android & Web

브라우저와 단말에서 댓글 분류·본문 요약을 수행해 서버 비용과 데이터 외부 전송을 줄인 1인 개발 프로젝트입니다.

React Native · Next.js · TypeScript · Transformers.js · WebLLM · Web Worker

[Android](#)
[Web](#)
[개발기](#)


액션독

게임 코어 엔진을 계층별로 설계하고 인증, 글로벌 랭킹, 광고까지 연결해 출시한 3라인 매치 게임입니다.

React Native · Expo · TypeScript · Supabase · AdMob

[Google Play](#)


HealthDog

운동 기록·조회·통계와 데이터 동기화, 구독 결제를 구현해 운영 중인 개인 프로젝트입니다.

Google OAuth · Firebase · Supabase · RevenueCat

[Google Play](#)

시스템 구성 상세

웹에서 구성요소를 선택해 확인하는 API·프론트·로컬 LLM의 상세 내용을 PDF용 정적 부록으로 옮겼습니다.

구현 상세 · 01

API

NestJS · TypeScript

5개 도메인의 동기 요청 처리, Worker가 위임하는 배치 실행까지 포함

구현

도메인 서비스가 데이터·외부 시스템 구현체를 직접 알지 않도록 포트와 어댑터로 분리했습니다. 외부 연동이 요청·수집·수신 세 방향이고 프로토콜이 제각각이라 하나로 억지 추상화하지 않고 각 구현을 어댑터로 격리했습니다. Worker가 호출하는 내부 배치 실행 엔드포인트도 같은 도메인 서비스를 재사용해, 실행 로직을 별도 앱으로 중복시키지 않았습니다.

구조

```
<모듈>/
├── adapter/
│   ├── controller/           HTTP 진입
│   └── dto/
├── application/
│   └── usecase/
│       └── <유스케이스>/
│           ├── *.usecase.ts   업무 기능 하나
│           ├── dto.ts
│           └── mapper.ts
└── infra/
    └── repository/           읽기 · 쓰기 분리
```

운영

요청 검증, 권한, 공통 오류 변환과 추적 정보를 한 경계에서 처리했습니다. 외부 오류는 내부 오류 형식으로 바꾸고 재시도 가능 여부를 구분했습니다.

출력

Repository 호출 · 로컬 LLM 호출 · 외부 연동 · Queue job · 배치 실행 결과

배포

API 컨테이너 수평 확장

입력

Web·PDA REST 요청 · Worker 내부 배치 실행 요청

계약

DTO validation · Symbol port · 공통 job package · 단 기 인증 토큰

설계 배경

초기 API는 Express로 자유도 높게 만들었고, 경계가 나뉘어 있지 않아 한 곳을 고치면 다른 곳이 깨졌습니다. NestJS와 클린 아키텍처를 도입해 모듈마다 방향을 고정했습니다.

더 읽기

이들 동안 운영 API가 다섯 번 죽었다

Web

Next.js · React · TypeScript

5개 업무 도메인의 90여 개 화면

구현

화면 단위 로컬 슬라이스와 공통 도메인 슬라이스를 분리하고, 서버 상태와 화면 상태의 책임을 나눴습니다. 기능이 많은 화면은 그 안에서 기능별로 다시 나눴습니다.

구조

```
<도메인 화면>/
├── _container/           화면 조합 · 상태 연결
├── _features/            업무 기능 묶음
│   └── <기능>/
│       ├── _container/   기능 상태 · 이벤트
│       ├── _view/        기능 표현
│       └── _hooks/        기능별 훅
├── _hooks/              화면 공통 훅
└── _store/              화면 상태
```

운영

화면과 기능이 폴더 단위로 갈려 있어 수정 범위가 그 폴더 안에 머뭅니다.

입력

사용자 입력 · 조회 조건 · 첨부 문서

출력

REST 요청 · 업무 결과 화면 · 파일 다운로드

계약

TypeScript DTO · HTTP status · 공통 오류 형식

배포

Web 컨테이너 독립 배포

설계 배경

프론트도 경계가 나뉘어 있지 않아 한 곳을 고치면 다른 화면이 깨졌습니다. API와 같은 방식으로 클린 아키텍처를 옮겨보려 했지만 상태 관리와 훅이 레이어 구분에 없지 않았습니다. 여러 구성을 시험한 끝에 기능 슬라이스로 나누는 FSD로 정리했습니다.

로컬 LLM

사내망 추론 서버 · 문서 추출 · OCR

문서 추출을 사내망 안에서 처리

구현

추론 서버를 사내망 장비에 띄우고 HTTP로 호출합니다. 고객사 문서라 망 밖으로 나갈 수 없어 외부 API를 쓰지 않았습니다. 추출 결과는 JSON Schema와 정규식으로 교차 검증합니다.

구조

텍스트 레이어 판정 · 스캔본 OCR(전체 페이지 → 표 영역 재인식) · schema parser · evaluation harness

운영

PDF에 텍스트 레이어가 있으면 그대로 읽고, 스캔본만 OCR로 처리합니다. 렌더 해상도는 실물 문서로 반복 측정해 정했습니다. 300dpi에서 나던 오인식이 400dpi에서 사라졌고 처리 시간은 거의 같았습니다.

입력

수출입 문서 · 추출 스키마

출력

검증된 구조화 데이터 · 실패 사유

계약

Internal HTTP · JSON Schema

배포

사내망 추론 서버 독립 운영